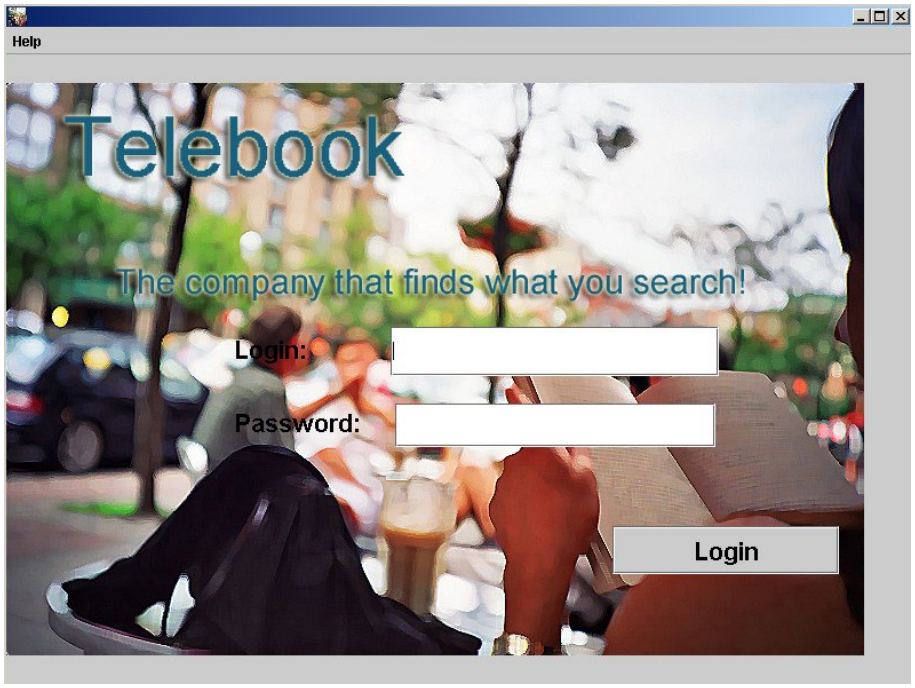


Team 1:

Tamer el Masri (0169854)
Collin Mous (0305774)
Sander Verweij (0305790)
Vikaash Mahabir (9621016)

Course: Database Project
Lecturer: Dr. Irina Neaga
Coordinator: Dr. Hamideh Afsarmanesh
Assistants: Mr. Gerben de Vries, Mr. Reinier Zevenhuizen

Table of Contents

Executive summary
1. Introduction Page 4
2. Literature review Page 5
3. Background of the project Page 5
3.1 Database design Page 5
3.2 Collection and analysis of requirements Page 6
3.3 System architecture Page 7
4. Description of the work Page 7
4.1 E-R Diagram Page 8
4.2 Tables Page 8
4.3 Relations between the entities Page 9
4.4 Data samples Page 10
4.5 Data views Page 10
5. Results Page 11
6. Scenario description Page 12
6.1 Scenario for adding an order Page 12
6.2 Scenario for adding a new book Page 13
7. Further extenions Page 13
8. Literature references Page 13

Annex 1. E-R Diagram Page 14
Annex 2. Screenshots of the interfaces Page 15
Annex 3. Java code Page 16

Executive Summary

In this paper we describe the proces of development considering a database bookstore application. We start by giving the background information about the project. After that we will give a collectiojn and analysis of the requirements followed by a description of the design phases. In the section after we will give a description about the work we have done and the choices made during the project. The results of the project will be given in the form of screenshots followed by a scenario description. In the conclusion we will discuss a few points that we wished to do in the project but we were not able to because of lack of time.

Team 1 Documentation of the bookstore database project: Telebook

Tamer el Masri,
Documentation, tmasri@science.uva.nl
Vikaash Mahabir,
Programming, vmahabir@science.uva.nl
Collin Mouse,
Design, cmous@science.uva.nl
Sander Verweij,
Programming, sverweij@science.uva.nl

This document presents a documentation of a database project focused on the development of a bookstore application. This project is done in the framework of a database project course at University of Amsterdam. The document contains a description of the main requirements, the architectural design, and the application carried out by the project group members during one month. This application is different from other applications mainly because the customers can be satisfied more efficiently by including their preferences in the database.

1. Introduction

Generally, a bookstore application is a database system which has the capabilities of storing and retrieving information about books, customers, and orders. In this project, we are considering to develop a sample centralized relational bookstore application database for the clerks and managers at a book store. This application may be further used for additional developments. This database can be used to store the records of customers and their preferences, the technical/non-technical books, magazines to which customers can be subscribed, and the customer-orders to the shop (e.g. telephone orders of customer for books, etc.), to be sent to their address for example, by regular mail.

2. Literature review

In this section we give a short description of the three-tier architecture.

In addition to the client-server kind of applications, some multi-threaded applications are conveniently modeled using three-tier architecture, also called client-agent-server architecture. In this architecture the client is only concerned with presentation services. The agent (or application server) processes the application logic for the client tier, hiding the underlying implementation and access details of the server tier and adding higher level support functionalities for the client. The server tier includes the low-level implementation of the data management services required by the agent tier

3. Background of the project

3.1 Database design

The design of the database consists of 3 phases.

1. The conceptual logical design phase
2. The physical design phase
3. The usage design phase
4. The test and evaluation phase

In the first phase the functional requirements, constraints and usage preferences are defined and modeled. A data model is also chosen and used to translate the requirements into the database schema.

In the second phase, the database management system is chosen and the storage structure is designed considering the database requirements and implementation using a database management system.

In the third phase, the access constraints and facilities, such as views, access rights and constraints are designed and implemented considering the database requirements.

3.2 Collection and analysis of requirements

3.2.1 The main development requirements for the database project

1. An ER model for the designed schema
2. Building a test database for a scenario/case study in order to be used for demonstration
3. Development of some basic functionalities
4. The development of a GUI for the users of this database (clerk and the manager).

3.2.2 Basic functionalities

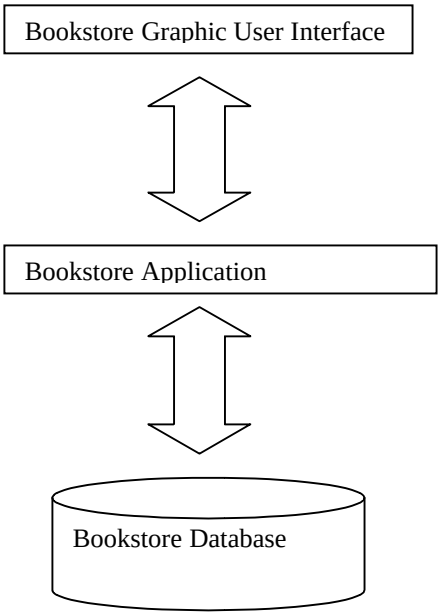
The ER diagram should consist of a minimum 4 entities that must at least each have one relationship with another entity in the schema and provide at least 3 attributes for each entity, and define a few integrity constraints.
The bookstore application server will be a Java program that connects to the bookstore database and for example, manipulates customers, books, and orders information using JDBC.

The minimum functionalities for manipulating the database entities include retrieve, add, update and delete each entity.

The GUI interface has to allow users (clerks and the manager) to access the bookstore database in order to perform the above operations.

3.3 System architecture

An overview of the system architecture for the project is illustrated in figure 1.



F

igure 1. Overview of the bookstore database application

The bookstore database is centralized and relational. It includes all the data related to the application, stored according to a database schema and manipulated by a relational database management system such as MAXDB and MySQL.

The bookstore application is the program, in Java, which connects to the database and manipulates the entities in the database.

The bookstore graphic user interface allows users to access the database in order to perform some operations such as add, delete and update.

4. Description of the work

In this chapter we will enlighten choices we have made considering our data and application

4.1 ER diagram

The ER diagram is represented in annex 1.

- Every customer in the database should be represented by his/her name, address and phone number. All these fields are mandatory for the customer in order to fill in the database Therefore; there will be no problems with contacting the customer if he/she has ordered something from the bookstore.
- An order consists of an order_id, a publication_id which represent the product that has been ordered (book or magazine) and a customer_id which identifies the customer that has placed the order. The order date is the date on which the order has been placed. This entity also represents a customer's history.
- An employee's function shows his/her position such as a clerk or a manager.
- Publication is a generalization of the entities of book and magazine. We made choice because book and magazine have some attributes in common.
- The primary key for book and magazine is the publication_id in combination with the ISBN for books and the issue for magazines
- Author has no relationship with magazine because magazines consist of articles written by different authors.
- The primary key for the entity preference is also customer_id but in combination with of type, which can be an author or a genre and it's value which is a author's name or the name of the genre.

4.2 Tables

Customer (customer_id, name, address, phonenumber)
Order (order_id, punlication_id, customer_id, order_date, delivery_date)
Employee (employee_id, function, address)
Publication (publication_id, title, price, year, publisher_id)
Publisher (publisher_id, name, address)
Author (author_id, name, genre)
Magazine (publication_id, issue, title, price, year, publisher_id)
Book (publication_id, ISBN, genre, author_id, title, price, year, publisher_id)
Preference (value, type, customer_id)

-

4.3 Relations between the entities

The relational schema is represented in figure 2.

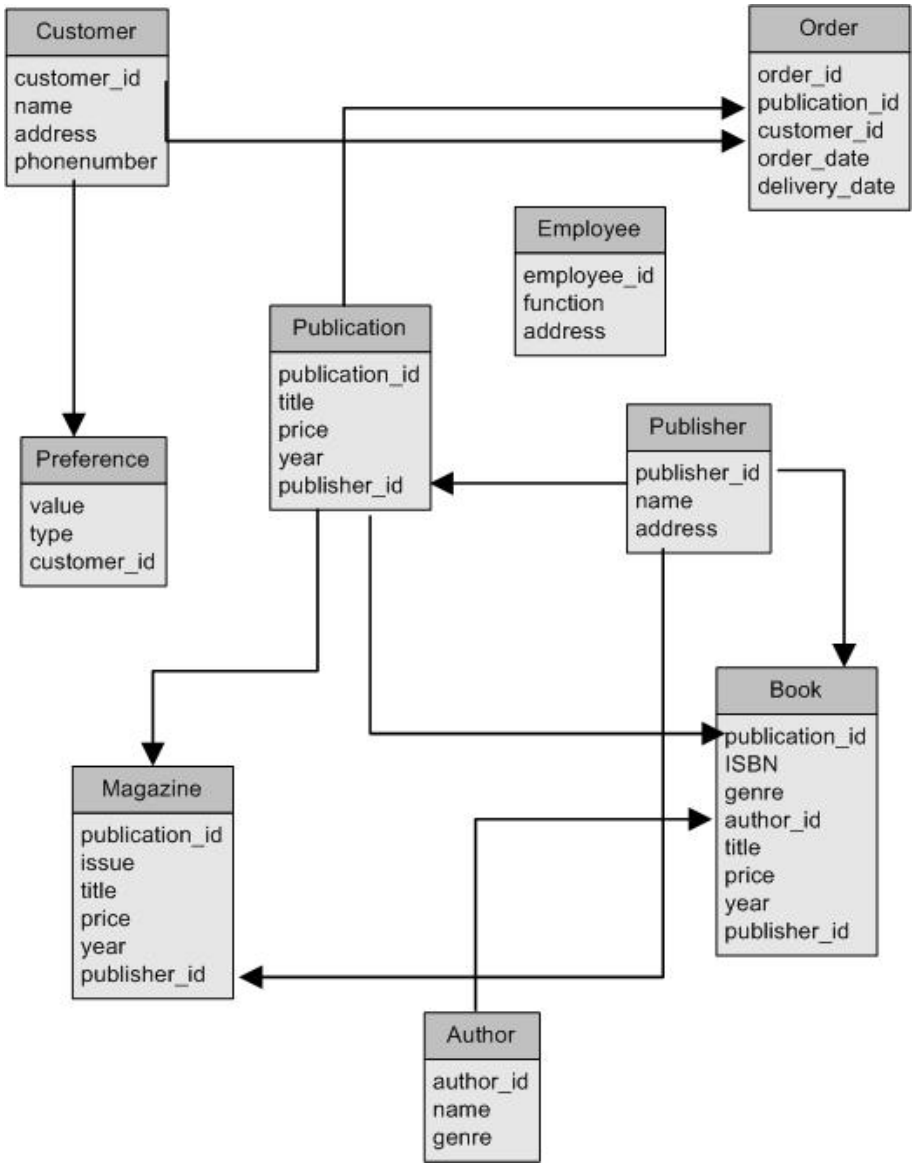


Figure 2, the relational schema

4.4 Data samples

In order to make a realistic application, we have used the current database bookstores to handle orders. The intention is not to recreate the current system, but to create a similar system which contains the strong points of the system currently used and adding new useful functionalities to this system. The reason behind this strategy is that the new system has all functionalities that a bookstore requires. However it has less flaws and new functionalities. This makes the project interesting for bookstores.

The data which we have entered in our sample database is realistic data acquired from queries entered in the system used in the bookstore. This information is publicly available and as we mentioned, our intention is to keep the system as realistic as possible. During the scenario and the demonstration we will try to make a role-play between a potential customer and a clerk. The scenario is similar to a situation that may occur in a bookstore.

4.5 Data views

It's important to distinguish and define two views. These views are as follows:

- manager view, concerning the manager and his tasks and rights;
- clerk view, concerning the rights and tasks of the clerk.

The intention has been to define the roles of the manager and the clerk, and to point out what is essential for both views to function.

The manager view is intended to be the view that has access to all rights. From our experience a bookstore is a company where managers and clerks are closely related to each other and can almost do the same work.

In a bookstore usually, the manager has a double function. The manager manages his staff, but at the same time, he's also a salesman. This brings us to the conclusion that the manager should have all rights, since he/she is performing in both functions at the same time. So not only can the manager use the manager functions in the developed GUI, which contain delete rights and modification rights to the customer and collection tables, but he is also able to place orders and add customers.

When we review the clerk function, we come to the conclusion to protect the database from inconsistency and personal gain or any circumstance alike, that the clerk does not have the abilities to modify and delete in the customer and collection tables. This means he cannot delete any records of customers or books, or modify them to his or her personal gain. In the past from our experience we have noticed that clerks sometimes modify customer records of friends to update their purchases or give them more discount then they might be allowed. To prevent these situations we restrict these rights from the clerk. The clerk is able to perform his function by taking orders by telephone and helping customers with their questions.

The manner in which we decide to achieve this is by adding an attribute in the employee table which has either the name "Manager" or "Clerk". This manner, when either a manager or a clerk logs in, this attribute will be checked (we intend to

build this in as a check in the code), and the correct view will be loaded. The distinction between the two views is that the manager has an extra tab in the GUI, in which he can perform the functions a clerk is restricted from.

5. Results

In this section we will show the screenshots of two interfaces. These are the interfaces which will be used to for the scenario descriptions in the next section.

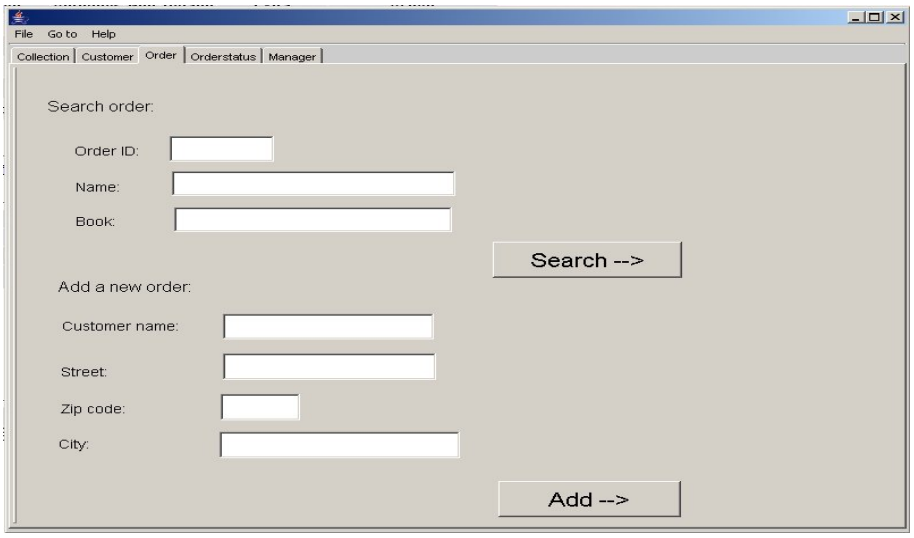


Figure 3. Interface for adding and searching of an order.

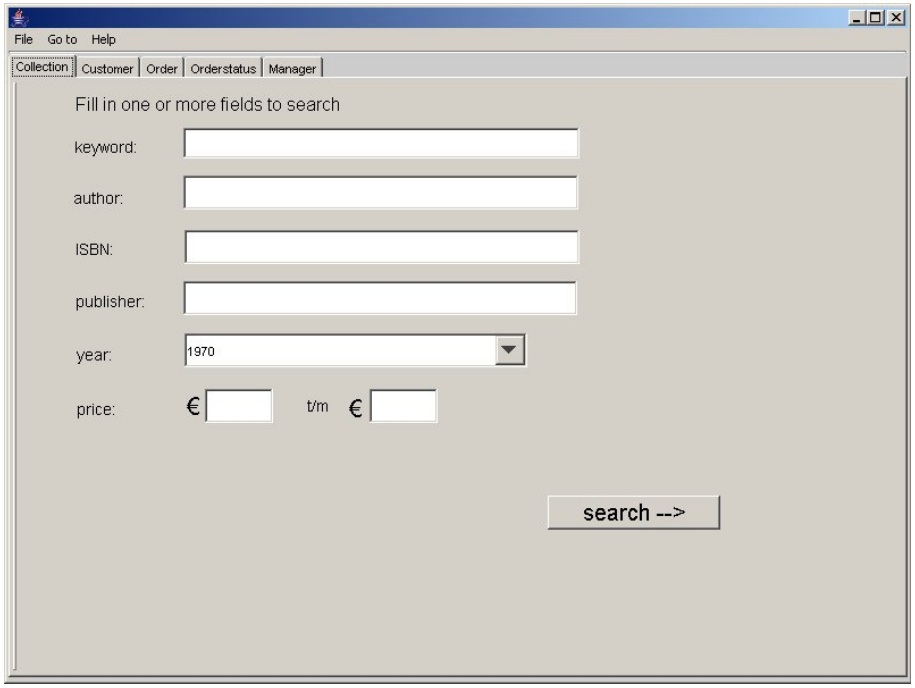


Figure 4. Interface for searching a book and a customer

6. Scenario description

6.1 Scenario for adding an order

- Clerk logs in to get access to the system using login and password.
- Customer calls the TELEBOOK bookstore to order a book.
- Clerk presses the *collection* tab.
- Clerk enters the title of the book in the screen and presses the search button.
- The results are shown in the collection screen.
- If the book is available, the clerk places the order by going to the order tab and fills in the order form.
- The system generates a order ID an gives it a order date.
- The Clerk fills in the customers name, address, phone number and a customer id is generated by the system.
- When the book is send to the customer the delivery date is entered into the system.
- If the book is not available a pop up is shown with the following message: book not in stock.
- If the book does not exists a pop up is show containing the following message: we don't sell this product.

6.2 Scenario for adding a new book

- The manager logs in into the system with his login and password
- The manager presses the manager tab (this is only visible for the manager and not for other employees)
- The manager presses the add tab
- The manager fills in the form containing the, Title, year, publisher id, price, ISBN number, author id.
- A publication ID will be generated.
- To check how the book is added to the system, the manager presses the collection tab
- In the collection tab the manager fills in the publisher id.
- The manager presses the search button.
- The result is shown in the collection screen otherwise the system gives a error message containing: ID is not valid.
- The manager logs out.

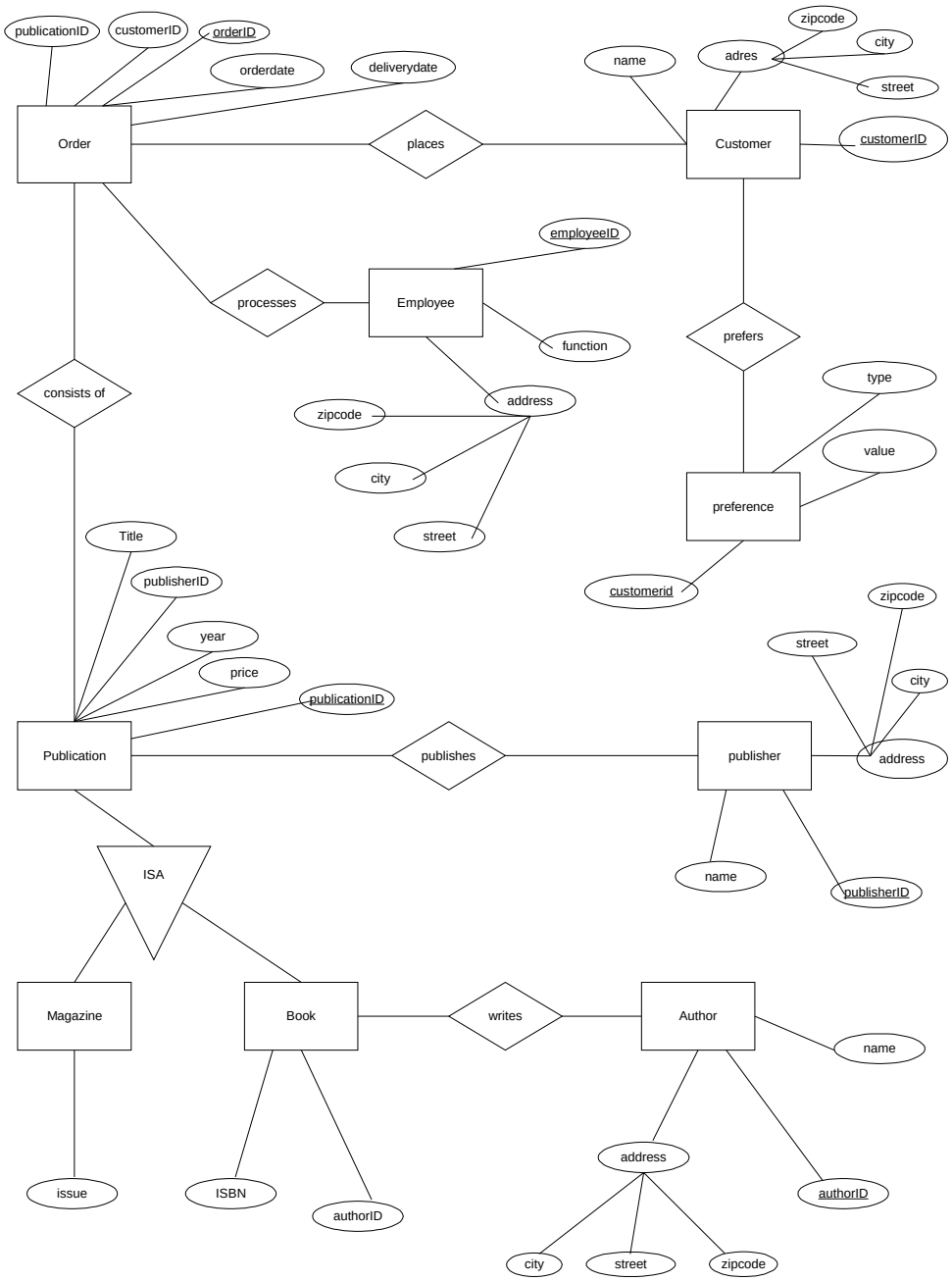
7. Further Extensions

- Up selling system, if a customer wants to order a book a screen disappears with other interesting titles, so the clerk can tell this to the customer. There is a big change the customer will also buy the other book, so their will be a profit for both parties.
- Administration system, Add a special tab for the manager where he can see how many books there were sold, the best employee, making a working schedule, see customers bills etc.
- Network implementation, for example communication with other shops, central database etc.

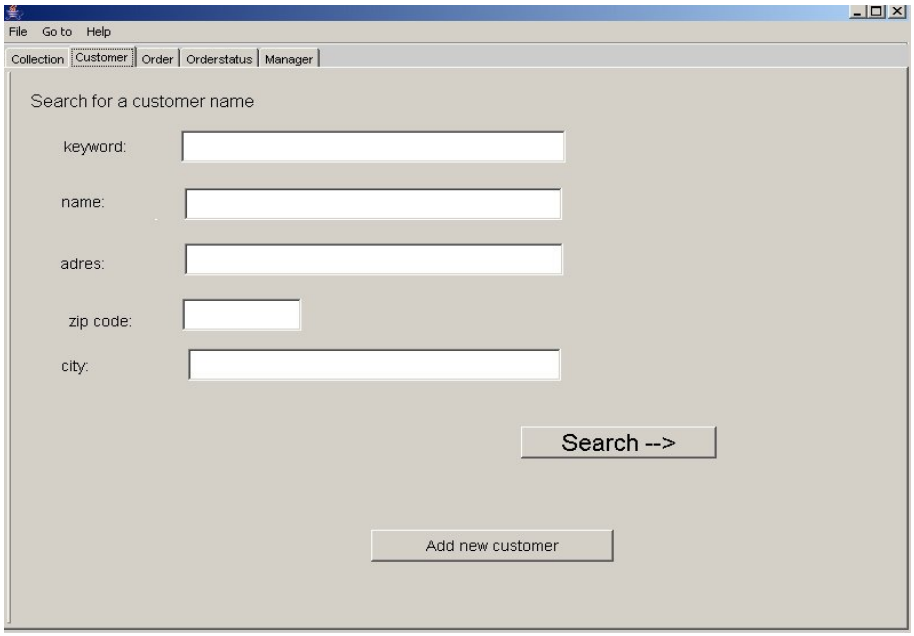
8. Literature references

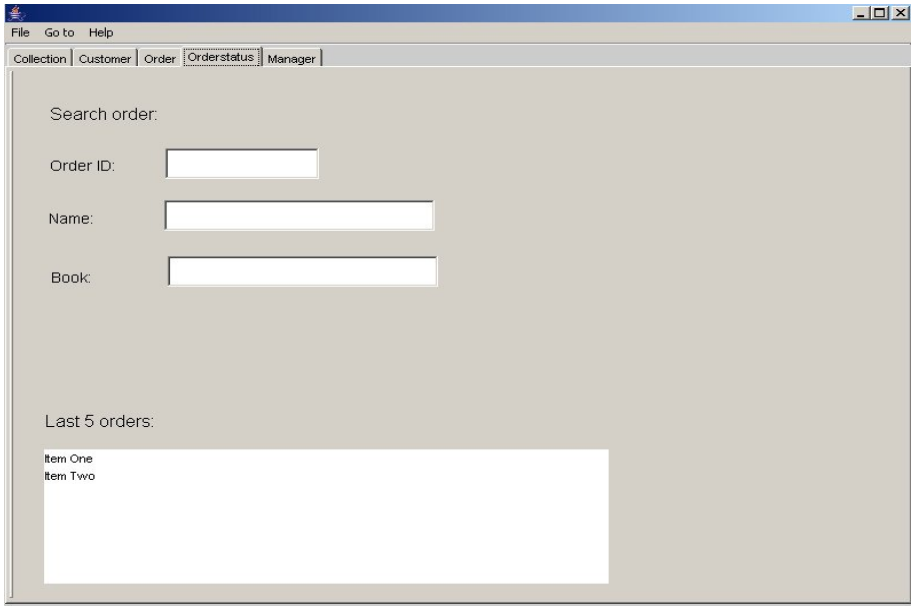
- Silberschatz, A,; Korth, H.F.; Sudarshan, S. Database System Concepts, 4th edition, 2002
- Deitel, H.M.; Deitel, P.J. Java How To Program, 4th edition
- Garita, C.; Ugur, Y.; Frenkel, A.; Afsarmanesh, H.; Hertzberger, L.O. DIMS: Implementation of a Federated Information Management System for PRODNET II

Annex 1. ER-diagram



Annex 2. Screenshots of the interfaces





Annex 3. Java code

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.ResultSetMetaData;
import java.sql.Statement;

/*
 * Created on Jun 22, 2005
 *
 * TODO To change the template for this generated file go to
 * Window - Preferences - Java - Code Style - Code Templates
 */

/**
 * @author vmahabir
 *
 * TODO To change the template for this generated type comment go to
 * Window - Preferences - Java - Code Style - Code Templates
 */
public class Sqlmethods {

	public static void main (String [] args) {
		//		CREATE CONNECTION
		try {
			Connection conn = Dbconnect.dbaseconnect();

			// CREATE STATEMENT
			Statement stmt;

			stmt = conn.createStatement();

			// VOER QUERY UIT
			/*String[] entiteiten = {"BOOK", "AUTHOR"};
			String[] query = {"BOOK.authorid",
"AUTHOR.authorid", "AUTHOR.name", "'Mak'"};
			StringBuffer buf = queryprocessor(stmt, query,
entiteiten);
			System.out.println(buf);*/

			//DOE EEN INVOER
			/*String[] insert = {"name", "Ernie"};
			int invoertest = insertfunction(stmt, insert,
"EMPLOYEE");
			System.out.println(invoertest);*/

			//DOE EEN DELETE
```

```
                        /*String[] delete = {"name", "Bernie", "function",
"Prutser"};
                        int deletetest = deletefunction(stmt, delete,
"EMPLOYEE");
                        System.out.println(deletetest);*/

                        //DOE EEN UPDATE
                        /*String[] wijzig = {"name", "Bernie", "function",
"Prutser"};
                        String[] waar = {"name", "Ernie"};
                        int updatetest = updatefunction(stmt, wijzig, waar,
"EMPLOYEE");
                        System.out.println(updatetest);*/

                        // SLUIT VERBINDING
                        conn.close();
                }
                catch (Exception e) {
                        System.out.println(e);
                }
        }
```

```
public static StringBuffer queryprocessor(Statement stmnt, String[] AttrAndValue,
String[] entiteit, boolean exactMatch) {

		// Does the user like to have exact or approximate results? Defaults
to approx.
		String comparison = " like ";

		if (exactMatch)
			comparison = " = ";

		//			BUILD QUERY
		// Where-clause
		String whereclause = null;
		for (int i = 0; i < AttrAndValue.length - 1; i = i + 2) {
			if (AttrAndValue[i] != null) {
				if (i == 0)
					whereclause = AttrAndValue[i] +
comparison + AttrAndValue[i+1];
				else
					whereclause += " AND " +
AttrAndValue[i] + comparison + AttrAndValue[i+1];
			}
		}

		//Entity-clause
		String entityclause = null;
		for (int i = 0; i < entiteit.length; i++) {
			if (entiteit[i] != null) {
				if (i == 0)
					entityclause = entiteit[i];
				else
					entityclause += ", " + entiteit[i];
			}
		}

		String searchstring = "SELECT * FROM " + entityclause + "
WHERE " + whereclause;
		System.out.println(searchstring);
		//		EXECUTE QUERY
		try {
			ResultSet results;
			results = stmnt.executeQuery(searchstring);


			//				GET ALL RESULTS
			StringBuffer buf = new StringBuffer();

			ResultSetMetaData rsmd = results.getMetaData();
```

```
                int numCols = rsmd.getColumnCount();
                int i, rowcount = 0;

                // get column header info
                for (i=1; i <= numCols; i++){
                        if (i > 1) buf.append(",");
                                buf.append(rsmd.getColumnLabel(i));
                }
                buf.append("\n");

                // break it off at 100 rows max
                while (results.next() && rowcount < 100){
                        // Loop through each column, getting the column
                        // data and displaying

                        for (i=1; i <= numCols; i++) {
                                if (i > 1) buf.append(",");
                                buf.append(results.getString(i));
                        }
                        buf.append("\n");
                        rowcount++;
                }
                results.close();
                return (buf);
        }
        catch (Exception e) {
                System.out.println(e);
                StringBuffer buf = null;
                return (buf);
        }
}
```

```
public static int insertfunction(Statement stmnt, String[] AttrAndValue, String
entiteit) {
                    //BUILD INSERTQUERY
                    String attributes = null;
                    String values = null;
                    for (int i = 0; i < AttrAndValue.length - 1; i = i + 2) {
                              if (AttrAndValue[i] != null) {
                                        if (i == 0) {
                                                  attributes = AttrAndValue[i];
                                                  values = """ + AttrAndValue[i+1] + """;
                                        }
                                        else {
                                                  attributes += ", " + AttrAndValue[i];
                                                  values += ", " + """ +
AttrAndValue[i+1] + """;
                                        }
                              }
                    }

                    String insertstring = "insert into " + entiteit + " (" + attributes + ")
values (" + values + ")";

                    //EXECUTE UPDATE
                    int results;
                    try {
                       results = stmnt.executeUpdate(insertstring);
                       return (results);
                    } catch (Exception e){
                       System.out.println(e);
                       results = 0;
                       return(results);
                    }
          }
```

```
public static int deletefunction(Statement stmnt, String[] AttrAndValue, String
entiteit) {
                        //BUILD DELETEQUERY
                        String whereclause = null;
                        for (int i = 0; i < AttrAndValue.length - 1; i = i + 2) {
                                if (AttrAndValue[i] != null) {
                                        if (i == 0)
                                                whereclause = AttrAndValue[i] + " = '"
+ AttrAndValue[i+1] + "'";
                                        else
                                                whereclause += " AND " +
AttrAndValue[i] + " = '" + AttrAndValue[i+1] + "'";
                                        }
                        }

                        String deletestring = "delete from " + entiteit + " where " +
whereclause;

                        //EXECUTE UPDATE
                        int results;
                        try {
                           results = stmnt.executeUpdate(deletestring);
                           return (results);
                        } catch (Exception e){
                           System.out.println(e);
                           results = 0;
                           return(results);
                        }
                }
```

```
public static int updatefunction(Statement stmnt, String[] InputAttrAndValue,
String[] AttrAndValue, String entiteit) {
                        //BUILD UPDATEQUERY (Set-clause)
                        String setclause = null;
                        for (int i = 0; i < InputAttrAndValue.length - 1; i = i + 2) {
                                if (InputAttrAndValue[i] != null) {
                                        if (i == 0)
                                                setclause = InputAttrAndValue[i] + " =
'" + InputAttrAndValue[i+1] + "'";
                                        else
                                                setclause += ", " +
InputAttrAndValue[i] + " = '" + InputAttrAndValue[i+1] + "'";
                                }
                        }

                        //BUILD UPDATEQUERY (Where-clause)
                        String whereclause = null;
                        for (int i = 0; i < AttrAndValue.length - 1; i = i + 2) {
                                if (AttrAndValue[i] != null) {
                                        if (i == 0)
                                                whereclause = AttrAndValue[i] + " = '"
+ AttrAndValue[i+1] + "'";
                                        else
                                                whereclause += " AND " +
AttrAndValue[i] + " = '" + AttrAndValue[i+1] + "'";
                                }
                        }

                        String updatestring = "update " + entiteit + " set " + setclause + "
where " + whereclause;

                        //EXECUTE UPDATE
                        int results;
                        try {
                           results = stmnt.executeUpdate(updatestring);
                           return (results);
                        } catch (Exception e){
                           System.out.println(e);
                           results = 0;
                           return(results);
                        }
                }
}
```